Практическая работа №2

Аудит готового клиент-серверного приложения

ФИО: Шипунов Даниил Станиславович

Цель работы: разобраться в устройстве готового приложения на Python, испытать передачу сообщений через TCP и UDP и оценить особенности форматов JSON и XML.

1 Ответы на вопросы

1. Какой протокол используется в send_reliable?
Функция send_reliable работает по TCP. На это указывают потоковый сокет SOCK_STREAM для IPv4 и установка соединения с помощью connect(). Такой обмен предусматривает сохранение порядка байтов и повторную передачу потерянных данных при работающем соединении.

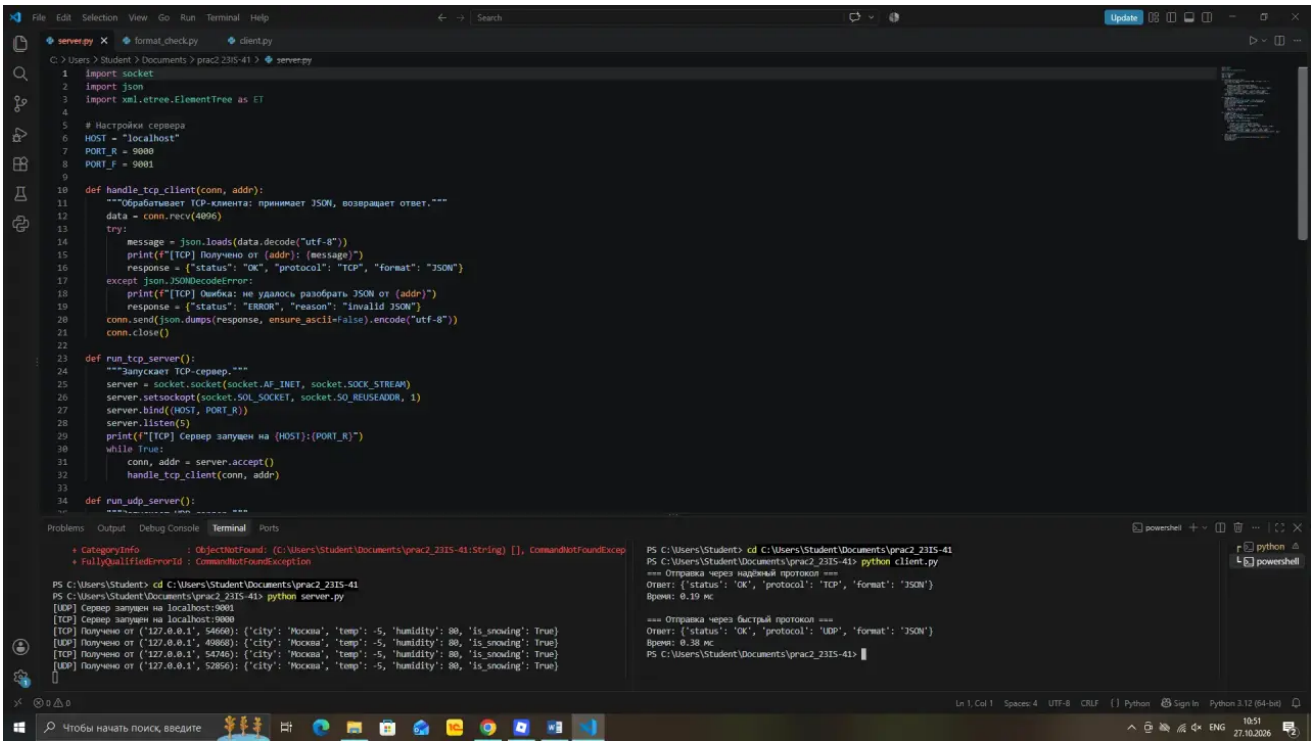
2. Какой протокол используется в send_fast?
В send_fast применяется UDP, поскольку для IPv4 выбран сокет SOCK_DGRAM. Передача выполняется вызовом sendto(), которому передаётся адрес получателя. Предварительное соединение не создаётся, а доставка и порядок датаграмм не гарантируются.

3. В каком формате передаются данные?
Клиент отправляет серверу JSON, представленный байтами UTF-8. Сначала json.dumps() сериализует данные, затем encode() подготавливает их для передачи через сокет. Ниже приведена строка, в которой выполняются эти преобразования:

```
client.send(json.dumps(message,
ensure_ascii=False).encode("utf-8"))
```

4. Что будет при отправке обычной строки hello?
Последовательность байтов b"hello" не является корректным JSON, поэтому её разбор завершится исключением JSONDecodeError. Обработчик сервера напечатает сообщение о невозможности разобрать JSON и отправит {"status": "ERROR", "reason": "invalid JSON"}. Затем соединение TCP закроется, но сам сервер останется доступен для следующих подключений.

2 Запуск сервера и клиента



На скриншоте показаны два терминала VS Code: в одном работает сервер, принимающий сведения о погоде через TCP и UDP, в другом выполнен клиент. Оба ответа имеют статус OK, а измеренное клиентом время составляет 0,19 мс при TCP и 0,38 мс при UDP.

3 Проверка граничных случаев

3.1 Сервер выключен и запущен TCP-клиент

Для проверки недоступного сервера его процесс остановлен, после чего запущен client.py. Попытка установить TCP-соединение привела к следующей ошибке:

```
ConnectionRefusedError:
[Errno 111] Connection refused
```

Причина отказа заключается в том, что на localhost:9000 никто не принимает подключения. Обработки этого исключения в клиенте нет, поэтому выполнение прекращается ещё до перехода к UDP-запросу.

3.2 Сервер выключен и запущен UDP-клиент

При остановленном сервере отдельно выполнена функция send_fast(weather). Она вернула {"status": "TIMEOUT"} спустя **2002,25 мс**, что соответствует ожиданию примерно в две секунды.

Отправка UDP-датаграммы сама по себе не подтверждает доступность адресата. Ответ не поступил за время, установленное settimeout(2.0), поэтому возник socket.timeout, а блок except преобразовал его в результат TIMEOUT.

3.3 Клиент отправляет не JSON

В открытое TCP-соединение с запущенным сервером передана последовательность b"hello" без сериализации в JSON. Такая отправка воспроизводит замену client.send() из задания и позволяет проверить реакцию сервера, не меняя исходный client.py.

```
{"status": "ERROR", "reason": "invalid JSON"}
```

В серверном журнале появилась запись «[TCP] Ошибка: не удалось разобрать JSON от ...». Полученное клиентом значение ERROR показывает, что некорректное сообщение распознано и обработано; завершения серверного процесса не произошло.

3.4 Сравнение форматов

Программа format_check.py сформировала обе записи и рассчитала длину их представления в UTF-8. При подсчёте учитываются все символы исходного вывода, включая пробелы JSON и теги XML.

Показатель	**JSON**	**XML**
Размер	72 байта	111 байт
Тип temp после чтения	int	str
Тип humidity после чтения	int	str
Тип is_snowing после чтения	bool	str

У JSON получилось 72 байта, у XML - 111 байт, поэтому разница составляет 39 байт в пользу JSON. Дополнительный объём XML создают парные теги; кроме того, при используемом способе чтения XML значения становятся строками, тогда как JSON сохраняет исходные типы.

4 Выявленные недостатки приложения

Сравнение результатов выполнено не полностью. Строка «Данные совпадают: True» выводится по результату проверки restored_json == weather. Результат XML с исходным словарём не сравнивается, хотя равенство restored_xml == weather ложно из-за строковых значений температуры, влажности и признака снега.

Обратное чтение XML фактически пропущено. Для restored_xml используются элементы дерева root, созданного до сериализации. Надёжнее заново прочитать строку xml_data через ET.fromstring(), преобразовать значения к нужным типам и только после этого проверять совпадение.

TCP-сообщение читается одним вызовом. Поток TCP может разделить переданный JSON на несколько частей, поэтому recv(4096) способен получить лишь фрагмент. Следует определить способ выделения полного сообщения, читать данные в цикле и отправлять их через sendall().

Ожидание TCP ничем не ограничено. В коде отсутствуют таймауты TCP и полноценная обработка сетевых сбоев. Поскольку обработчики подключений выполняются по очереди, клиент без отправленных данных может задержать обслуживание всех следующих TCP-подключений.

Набор перехватываемых исключений слишком узок. Сервер умеет обрабатывать JSONDecodeError, однако ошибки декодирования UTF-8 и операций сокета остаются без обработки. Например, UnicodeDecodeError может прервать текущий серверный цикл; этот риск выявлен при анализе кода, без отдельного испытания повреждённых байтов.

Размер буфера требует учёта. Для UDP указан буфер приёма 4096 байт, и превышающая его датаграмма может поступить в усечённом виде. Для TCP число 4096 ограничивает один recv(), поэтому большие сообщения требуют нескольких чтений и сборки полученных частей.

Содержимое объекта weather не проверяется. Успешный разбор JSON ещё не означает, что сообщение содержит правильные сведения о погоде. Нужны отдельные проверки полей city, temp, humidity и is_snowing, их типов и допустимых значений.

5 Вывод

Для передачи этих сведений я предпочёл бы сочетание **TCP и JSON**. TCP подходит для обмена, в котором важны доставка и правильная последовательность данных, но на уровне программы необходимо дополнительно предусмотреть таймауты, разрыв соединения и выделение целых сообщений.

Выбор JSON объясняется меньшим размером записи и сохранением числовых и логических типов при восстановлении объекта. UDP можно использовать для непрерывного потока обновлений, если пропуск отдельного обновления допустим. Проверка показала работоспособность штатного обмена и реакции на ошибочный JSON, а анализ кода выявил возможности для повышения устойчивости приложения.